

# Transformers Evolution from BERT to GPT, T5, and beyond...

Evolution from BERT to  
GPT, T5, and Beyond

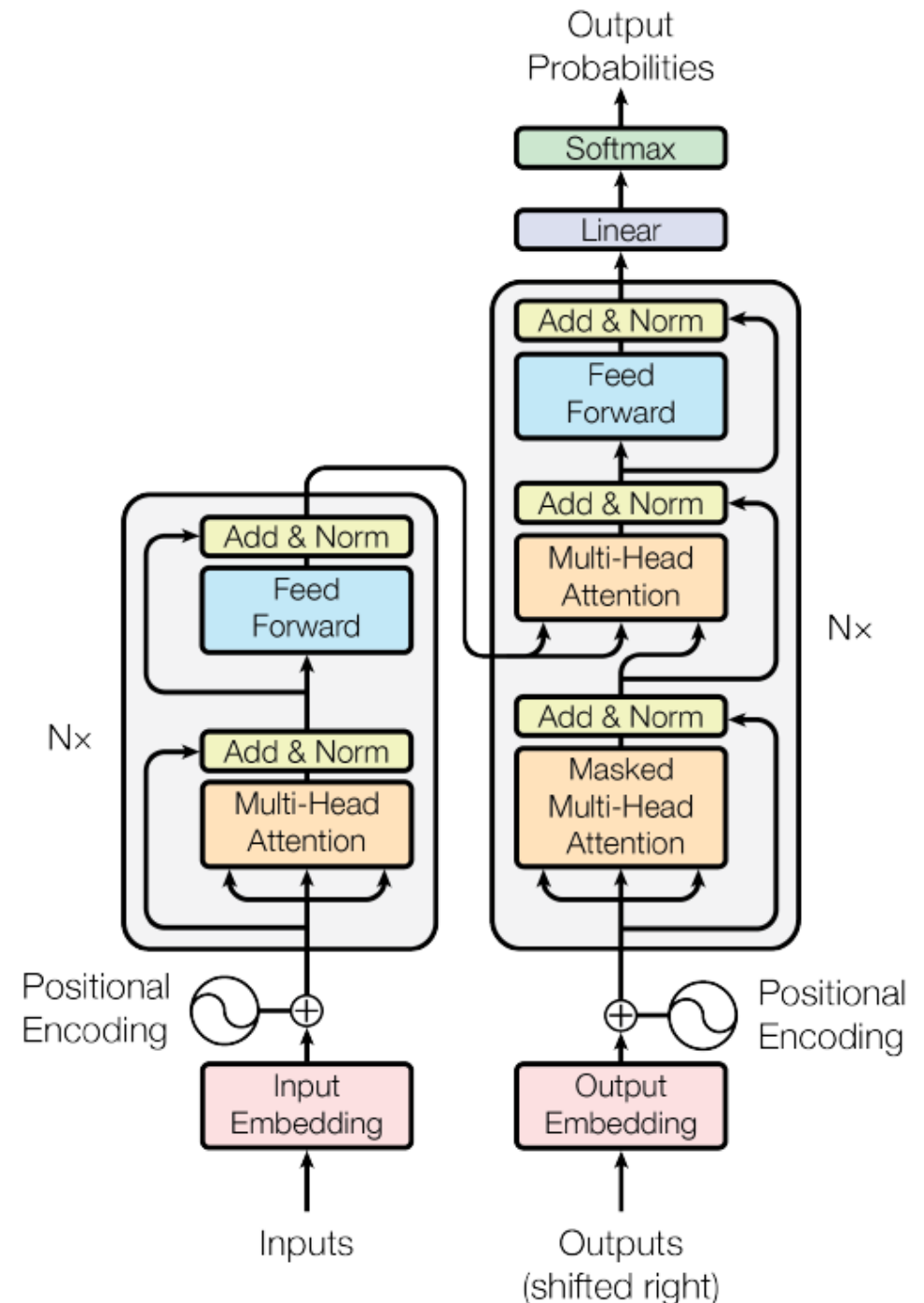
Sanjaya Edirisinghe  
[sanjayae@bu.edu](mailto:sanjayae@bu.edu)



# Transformers Evolution from BERT to GPT, T5, and beyond...

## CONTENT :

1. Why Transformers ?
2. Transformer Model Parameters
3. Test scores of AI systems on various capabilities
4. Benchmark Scores Comparison
5. Research Areas for (Gen) AI field
6. Application of AI Models
7. Model Fine-Tuning
8. Types of Fine-Tuning

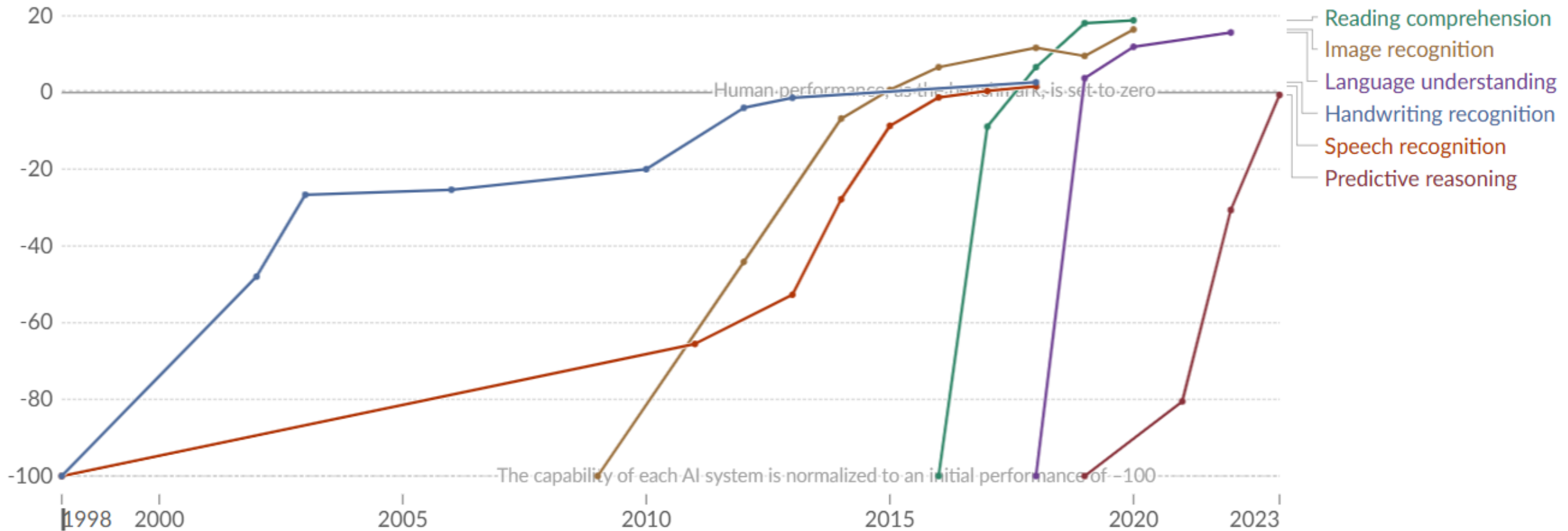


# Transformer Model Parameters

| Application Name                       | Developer                                  | Year | Number of Parameters                  |
|----------------------------------------|--------------------------------------------|------|---------------------------------------|
| GPT-3                                  | OpenAI                                     | 2020 | 175 billion                           |
| GPT-4                                  | OpenAI                                     | 2023 | Undisclosed (estimated ~1.7 trillion) |
| T5 (Text-To-Text Transfer Transformer) | Google Brain                               | 2019 | 77 million to 11 billion              |
| Switch Transformer                     | Google                                     | 2021 | 1.6 trillion                          |
| Wu Dao 2.0                             | Beijing Academy of Artificial Intelligence | 2021 | 1.75 trillion                         |
| LLaMA                                  | Meta AI                                    | 2023 | 7 billion to 65 billion               |
| LLaMA 2                                | Meta AI                                    | 2023 | 7 billion to 70 billion               |
| Gemini 1                               | Google DeepMind                            | 2023 | 1.8 billion and 3.25 billion          |
| Gemini 1.5 Pro                         | Google DeepMind                            | 2024 | Multimodal sparse mixture-of-experts  |
| Gemini 2                               | Google DeepMind                            | 2024 | 2 billion, 9 billion, and 27 billion  |

| Model Name                                                                                                          | Architecture                                                                                                                | Key Features                                                                                                                                                                                     | Strengths                                                                                  | Limitations                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>BERT</b> (Bidirectional Encoder Representations from Transformers)<br><br>Developed by Google AI (2018)          | 1- Encoder-only model.<br><br>2- It uses a bidirectional approach to understand context from both left and right of a word. | 1- Masked Language Modeling (MLM): Randomly masks words in a sentence and predicts them.<br><br>2- Next Sentence Prediction (NSP): Predicts if one sentence follows another.                     | 1- Requiring contextual understanding (e.g sentiment analysis.)                            | 1- Encoder-only<br><br>2- Not ideal for generative tasks like text generation.<br><br>3 - Fixed input size. |
| <b>GPT</b> (Generative Pre-trained Transformer):<br><br>By OpenAI (GPT-1 in 2018, GPT-2 in 2020, GPT-3 in 2023).4.x | 1 - Decoder-only model<br><br>2- Uses unidirectional attention, processing tokens <b>sequentially</b> to generate text.     | 1- Pre-trained on massive datasets with autoregressive objectives (predicts the next word given previous words).<br><br>2- Powerful for generative tasks (e.g., text generation, summarization). | 1- Handles long-form text generation.<br>2 -Effective in few-shot and zero-shot learning.  | 1- Struggles with bidirectional understanding.<br><br>2- Requires prompt engineering for optimal results.   |
| <b>T5</b> (Text-to-Text Transfer Transformer):<br><br>By Google AI in 2020.                                         | Encoder-decoder model. Treats all NLP tasks as text-to-text tasks.                                                          | 1- Converts inputs and outputs into text (e.g. Language Translation)<br>2. Text summarization<br>3. Text classification.                                                                         | 1- Unified framework for diverse NLP tasks.<br>2- Highly versatile in multi-task learning. | Computationally intensive compared to encoder-only or decoder-only models.                                  |

# Test scores of AI systems on various capabilities relative to human performance



# Benchmark Scores Comparison

| Model               | Average | Multilingual | Tool Use | Math   | Reasoning | Code   | General |
|---------------------|---------|--------------|----------|--------|-----------|--------|---------|
| Claude 3.5 Sonnet   | 82.10%  | 91.60%       | 90.20%   | 71.10% | 59.40%    | 92.00% | 88.30%  |
| GPT-4o              | 80.53%  | 90.50%       | 83.59%   | 76.60% | 53.60%    | 90.20% | 88.70%  |
| Meta Llama 3.1 405b | 80.43%  | 91.60%       | 88.50%   | 73.80% | 51.10%    | 89.00% | 88.60%  |
| GPT-T Latest        | 78.12%  | 88.50%       | 86.00%   | 72.60% | 48%       | 87.10% | 86.50%  |
| Claude 3 Opus       | 76.70%  | 90.70%       | 88.40%   | 60.10% | 50.40%    | 84.90% | 85.70%  |
| OpenAI GPT-4        | 75.52%  | 85.90%       | 88.30%   | 64.50% | 41.40%    | 86.60% | 86.40%  |

<https://www.vellum.ai/blog/llm-benchmarks-overview-limits-and-model-comparison>

# Research Areas for (Gen) AI field

## **Effectiveness**

- Reasoning Capabilities (Decision making with logical reasoning)
- Meaningful context (Context awareness, long-range dependencies)
- Explainable GEN AI (Transparency)

## **Efficiency improvement:**

- More efficient AI architecture (Low watt per token)
- “The bigger the better!” should not be true for model training and constriction of foundation models
- Training models with small number of data sets
- Multi-modal capabilities

# Research Areas for (Gen) AI field ctd..

## **Responsible AI**

- AI Ethics
- Un-bias/fair AI models
- Explainable AI (Transparency)

## **Regulating AI**

- Regulatory framework (NIST AI 600-1), EU AI Act

## **AI safeguards tools:**

- Content moderation tools: Filtering AI-generated content that violates terms of service.
- AI detection algorithms: Identifying text, images, or audio created by AI systems.
- Transparency requirements: Forcing AI developers to disclose how their systems work to increase accountability.

# Application of AI Models

## 01 Text Classification, sentiment analysis

```
1 from transformers import pipeline
2
3 classifier = pipeline(task="text-classification", model="bert-base-uncased")
4 result = classifier("I love this product! It's amazing.")
5 print(result)
6
```

# Application of AI Models

## 02. Text Classification, sentiment analysis

```
10 from transformers import pipeline
11
12 qa = pipeline(task="question-answering", model="bert-large-uncased-whole-word-masking-finetuned-squad")
13 context = "Artificial Intelligence is the simulation of human intelligence in machines."
14 question = "What is Artificial Intelligence?"
15 result = qa(question=question, context=context)
16 print(result)
17
```

# Application of AI Models

## 03. Text summarization

```
19 from transformers import pipeline
20
21 summarizer = pipeline(task="summarization", model="t5-small")
22 text = ("Natural Language Processing enables computers to understand human language."
23         " Applications include chatbots, machine translation, and sentiment analysis.")
24 result = summarizer(text, max_length=30, min_length=10, do_sample=False)
25 print(result)
26
```

# Model Fine-Tuning

## Fine-Tuning : Transfer Learning for Domain-Specific Tasks

- Adapting a pre-trained model to a specific task or domain using a smaller labelled dataset.
- Leverage general knowledge from pre-training while learning domain-specific nuances.

### Select Pre-Trained Model

- Choose a model based on task requirements
- e.g., BERT for classification, T5 for summarization

### Prepare Data set

- Collect a labeled dataset (e.g., medical records)
- Preprocess the text (tokenization, normalization).

### Model Training (Supervised Learning:)

- Add a task-specific layer (e.g., classification head) to the model.
- Train the model on the labeled dataset.

### Evaluate and Optimize

- Use metrics: "accuracy", F1 score, "BLEU to evaluate performance
- Optimize hyperparameters for better results

# Types of Fine-Tuning:

## 1. Full Fine-Tuning:

- Update all model weights
- Best for large datasets.

## 2. Feature-Based Fine-Tuning:

- Freeze pre-trained layers and only update task-specific layers.
- Useful for small datasets.

## 3. Parameter-Efficient Fine-Tuning:

- Techniques like LoRA (Low-Rank Adaptation) or Adapters modify only a subset of model parameters.